Artificial Intelligence

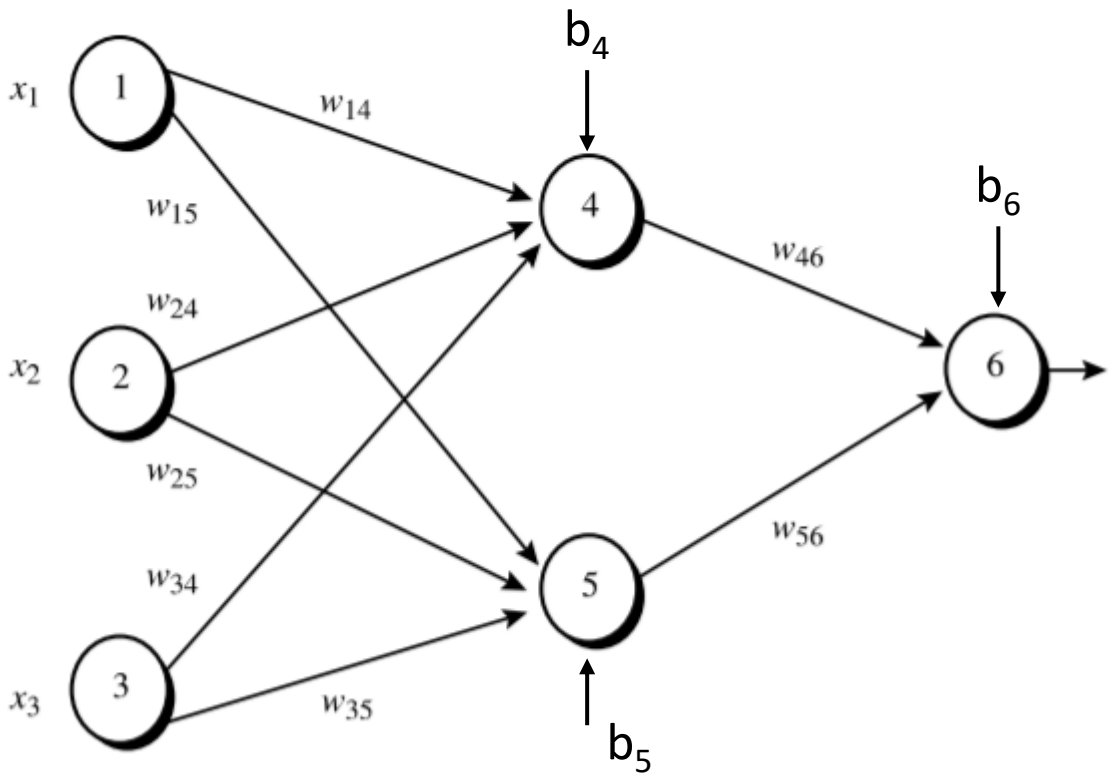
Neural Network

Artificial Neural Network Learning

- <u>How does a perceptron learn the appropriate weights?</u>

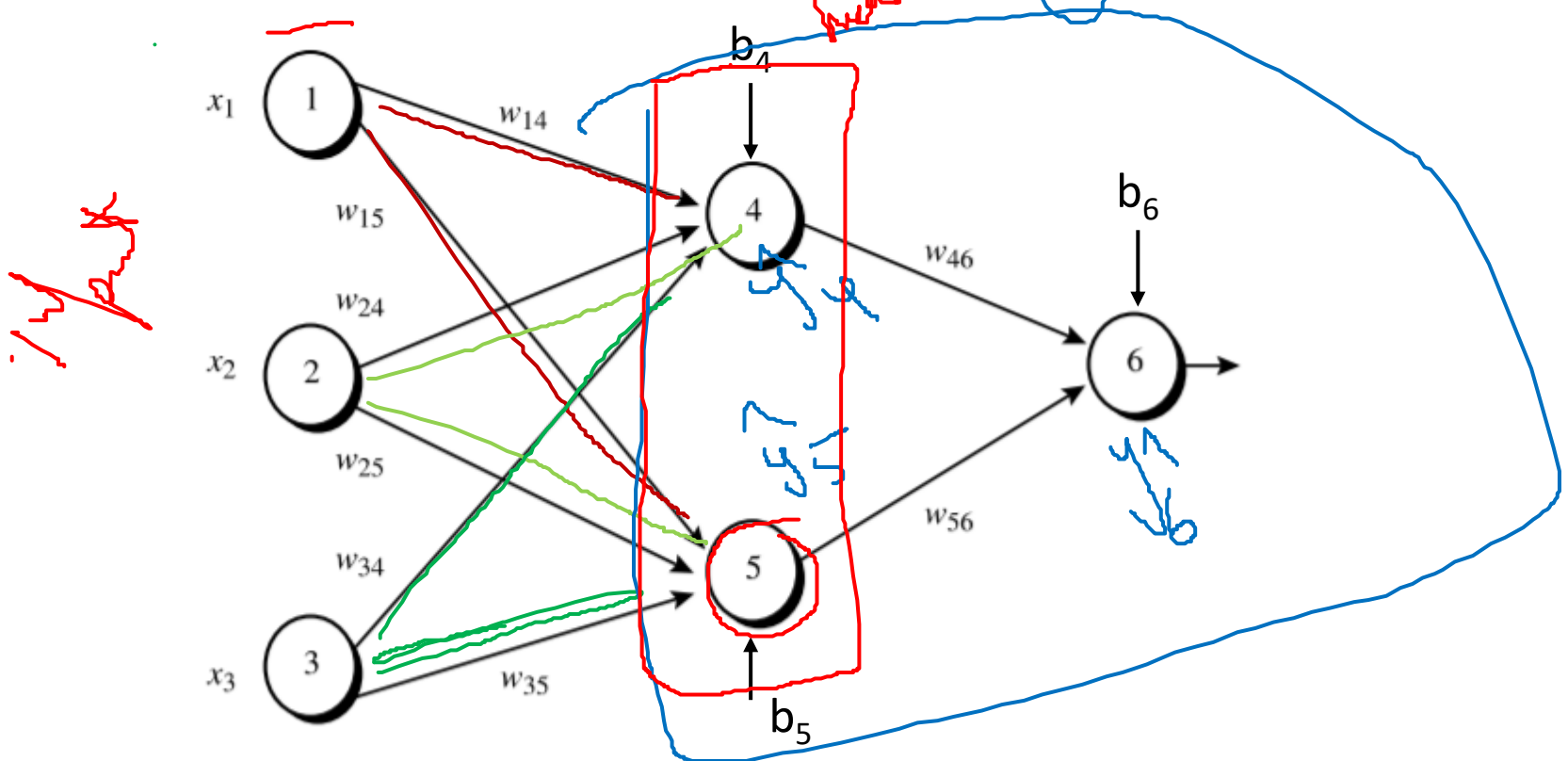
Example

x1	x2	x3	w14	w15	w24	w25	w34	w35	w46	w56	b4	b5	b6
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1



Example

x1	x2	x3	w14	w15	w24	w25	w34	w35	w46	w56	b4	b5	b6
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1



Feed-Forward

$$\sigma = \frac{1}{1+e^{-y}}$$

$$y_4 = (x_1 \cdot w_{14} + x_2 \cdot w_{24} + x_3 \cdot w_{34}) + b_4$$

$$= 1 \cdot 0.2 + 0 \cdot 0.4 + 1 \cdot (-0.5) + (-0.4)$$

$$\hat{y}_4 = \sigma(-0.7) = 0.332$$

$$y_5 = x_1 \cdot w_{15} + x_2 \cdot w_{25} + x_3 \cdot w_{35} + b_5$$

$$= -0.1$$

$$\hat{y}_5 = \sigma(-0.1) = 0.524$$

$$y_6 = \hat{y}_4 \cdot w_{46} + \hat{y}_5 \cdot w_{56} + b_6$$

$$= (0.00 \quad 44)$$

$$\hat{y}_6 = 0.5$$

Backpropagation

- Error = ½ (Target – output)^2 =>**Squared Error Function**
- Derivate of Squared Error Function= (Target – output)

- Sigmoid Function: σ
- Derivate of Sigmoid Function: σ (1-σ)

Backpropagation

- Error = ½ (Target – output)^2 =>**Squared Error Function**
- Derivate of Squared Error Function= (Target – output)

- Sigmoid Function: σ
- Derivate of Sigmoid Function: σ (1-σ)

Review: Partial Derivate Example

Rule	$f(x)$	Scalar derivative notation with respect to x	Example
Constant	c	0	$\frac{d}{dx}99 = 0$
Multiplication by constant	cf	$c\frac{df}{dx}$	$\frac{d}{dx}3x = 3$
Power Rule	x^n	nx^{n-1}	$\frac{d}{dx}x^3 = 3x^2$
Sum Rule	$f + g$	$\frac{df}{dx} + \frac{dg}{dx}$	$\frac{d}{dx}(x^2 + 3x) = 2x + 3$
Difference Rule	$f - g$	$\frac{df}{dx} - \frac{dg}{dx}$	$\frac{d}{dx}(x^2 - 3x) = 2x - 3$
Product Rule	fg	$f\frac{dg}{dx} + \frac{df}{dx}g$	$\frac{d}{dx}x^2x = x^2 + x2x = 3x^2$
Chain Rule	$f(g(x))$	$\frac{df(u)}{du}\frac{du}{dx}$, let $u = g(x)$	$\frac{d}{dx}ln(x^2) = \frac{1}{x^2}2x = \frac{2}{x}$

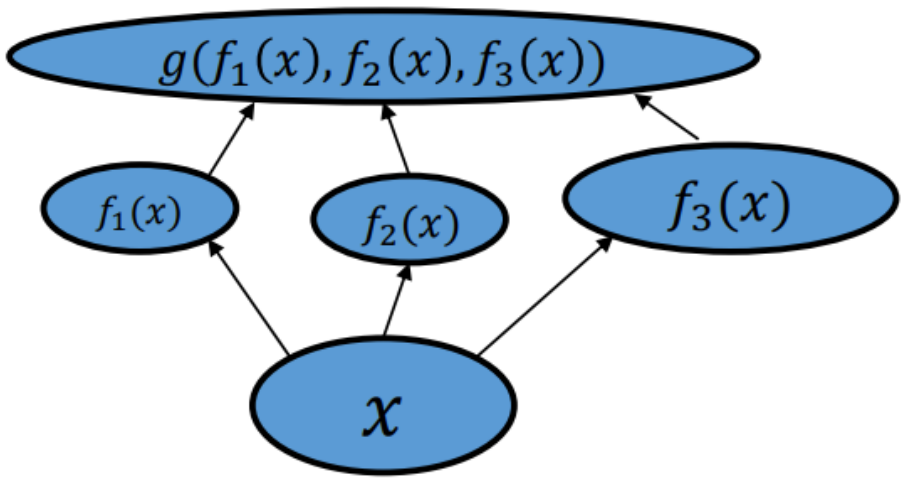
Review: Chain-Rule

- Univariate Chain Rule

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

- Multivariate Chain Rule

$$\frac{\partial g}{\partial x} = \sum \frac{\partial g}{\partial f_i}\frac{\partial f_i}{\partial x}$$



Compute Gradient

Output layer

$$Err_j = O_j(1 - O_j)(T_j - O_j)$$

Hidden layer

$$Err_j = O_j(1 - O_j)\sum_k Err_k w_{jk},$$

$$Error\ at\ unit\ 6 = \sigma(1 - \sigma)(T - \hat{y}_6\ )$$

Error at unit 6= $\hat{y}_6\ \left(1 - \hat{y}_6\ \right)(1 - \hat{y}_6\ )$

0.5 (1-0.5)(1-0.5) = 0.125

Error at unit 6

ΔE_6

$$Err\ at\ unit\ 5 = \frac{\partial E}{\partial \hat{y}_6} \cdot \frac{\partial \hat{y}_6}{\partial y_6} \cdot \frac{\partial y_6}{\partial \hat{y}_5} \cdot \frac{\partial \hat{y}_5}{\partial y_5}$$

ΔE_5

$$\hat{y}_5(1 - \hat{y}_5)\Delta E_6 \omega_{56}$$

Error at unit 5

-0.0349

$$Err\ at\ unit\ 4 = \frac{\partial E}{\partial \hat{y}_6} \cdot \frac{\partial \hat{y}_6}{\partial y_6} \cdot \frac{\partial y_6}{\partial \hat{y}_4} \cdot \frac{\partial \hat{y}_4}{\partial y_4}$$

ΔE_4

$$\hat{y}_4(1 - \hat{y}_4)\Delta E_6 \omega_{46}$$

Error at unit 4

- 0.0165

Weights Update

$$\Delta w_{ij} = \eta \delta_j o_i$$

$\eta = 0.01$

ΔE_6. (input at unit 6 or Output unit 5)

$\omega_{56(\text{new})} = \omega_{56} + \eta \ \Delta E_6 \hat{y}_5 = \cancel{-0.195} \ -0.197$

$\omega_{46(\text{new})} = \omega_{46} + \eta \ \Delta E_6 \hat{y}_4 = -0.298$

$\omega_{14\ (\text{new})} = \omega_{14} + \eta \cdot \Delta E_4 \cdot (x_1)$

$\omega_{15(\text{new})} = \omega_{15} + \eta \cdot \Delta E_5 \ (x_1)$

$\omega_{24(\text{new})} = \omega_{24} + \Delta E_4 x_2$

$\omega_{34(\text{new})} = \omega_{34} + \Delta E_4 x_3$

$\omega_{25(\text{new})} = \omega_{25} + \Delta E_5 x_2$

$\omega_{35(\text{new})} = \omega_{35} + \Delta E_5 x_3$

Weights Update

$\eta = 0.01$

ΔE_6. (input at unit 6 and Output unit 5)

$\omega_{56} = \omega_{56} + \eta \quad \Delta\omega_{56} = \cancel{-0.195} \; -0.197$

$\omega_{46} = \omega_{46} + \eta \quad \Delta\omega_{46} = -0.298$

$\omega_{14} = \omega_{14} \times \eta \cdot \Delta E_4 \cdot (x_1)$

$\omega_{15} = \omega_{15} \times \eta \cdot \Delta E_5 \cdot (x_1)$

$\omega_{24} = \omega_{24} + \Delta E_4 x_2$

$\omega_{34} = \omega_{34} + \Delta E_4 x_3$

$\omega_{25} = \omega_{25} + \Delta E_5 x_2$

$\omega_{35} = \omega_{35} + \Delta E_5 x_3$

Bias Update

- $b_6 = b_6 + \eta . \Delta b_6$ $\Rightarrow \Delta b_6$ = ΔError at unit 6

- $b_5 = b_5 + \eta . \Delta b_5$

  ΔE_5

- $b_4 = b_4 + \eta . \Delta b_4$

  ΔE_4

1. Compute gradients

$$Err_j = O_j(1 - O_j)(T_j - O_j)$$

$$Err_j = O_j(1 - O_j)\sum_k Err_k w_{jk},$$

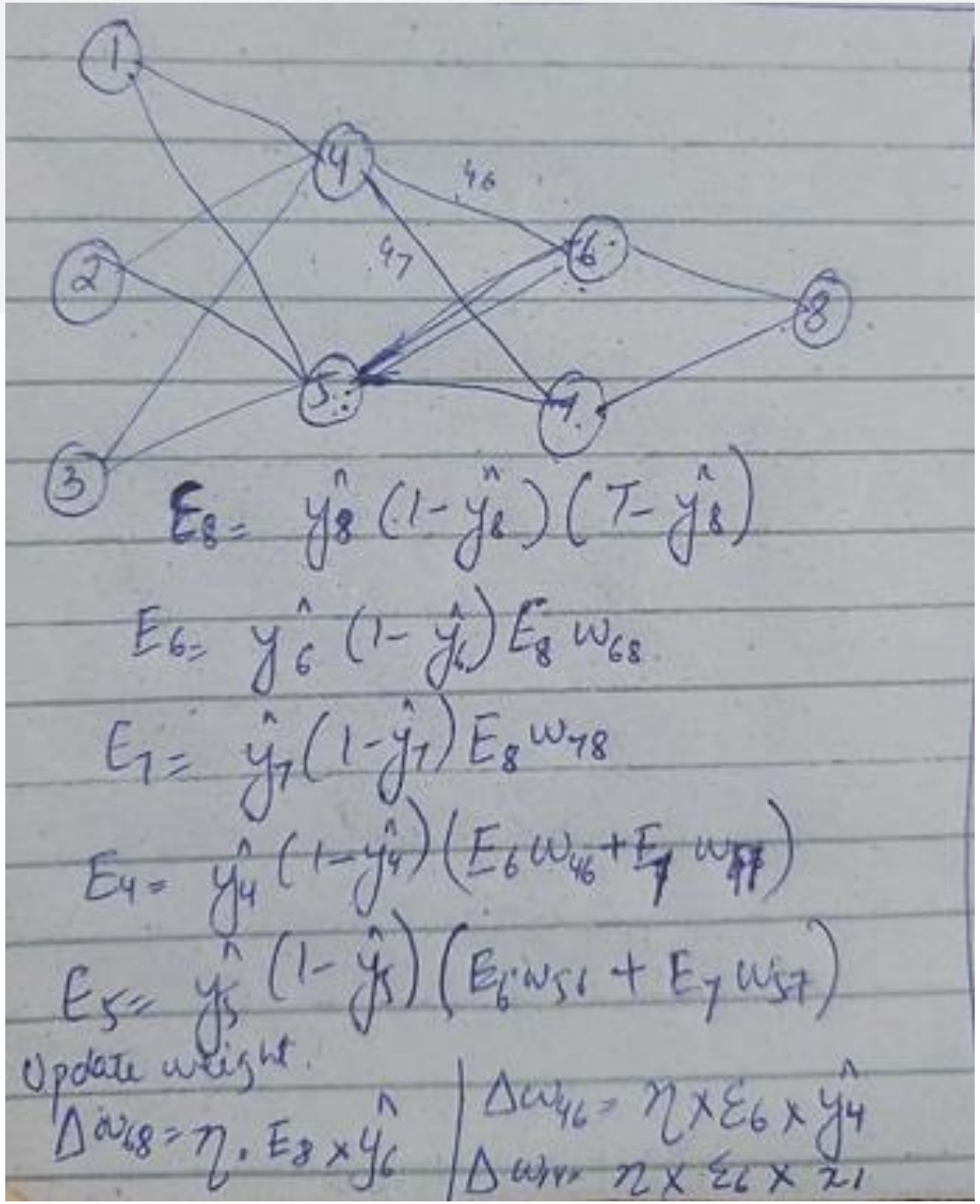
2. Update weights

$w_{new} = w_{old} + \Delta w_{ij}$

$$\Delta w_{ij} = \eta \delta_j o_i$$

3. Update Bias

$b_n = b_n + \eta . \Delta b_n \quad \Rightarrow \Delta b_n = \Delta E$rror at unit n



1. Compute gradients

$$Err_j = O_j(1 - O_j)(T_j - O_j),$$

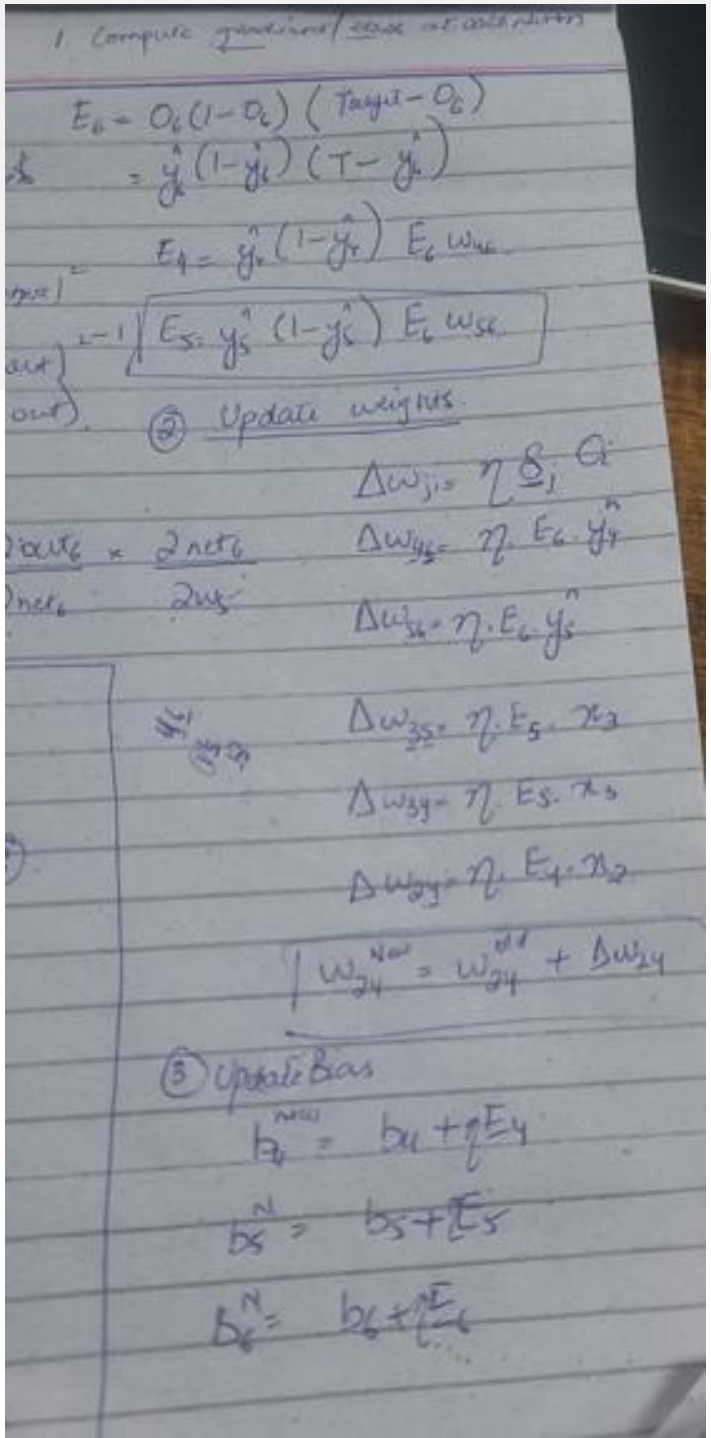
$$Err_j = O_j(1 - O_j)\sum_k Err_k w_{jk},$$

2. Update weights

$w_{new} = w_{old} + \Delta w_{ij}$

3. Update Bias

$b_n = b_n + \eta . \Delta b_n \qquad \Rightarrow \Delta b_n$ = ΔError at unit n



Learning Rate

- This is a subtle art.
- Too small: can take days instead of minutes to converge
- Too large: diverges (MSE gets larger and larger while the weights increase and usually oscillate)
- Sometimes the “just right” value is hard to find.

Train ANN

Training neural nets:

Loop until convergence:

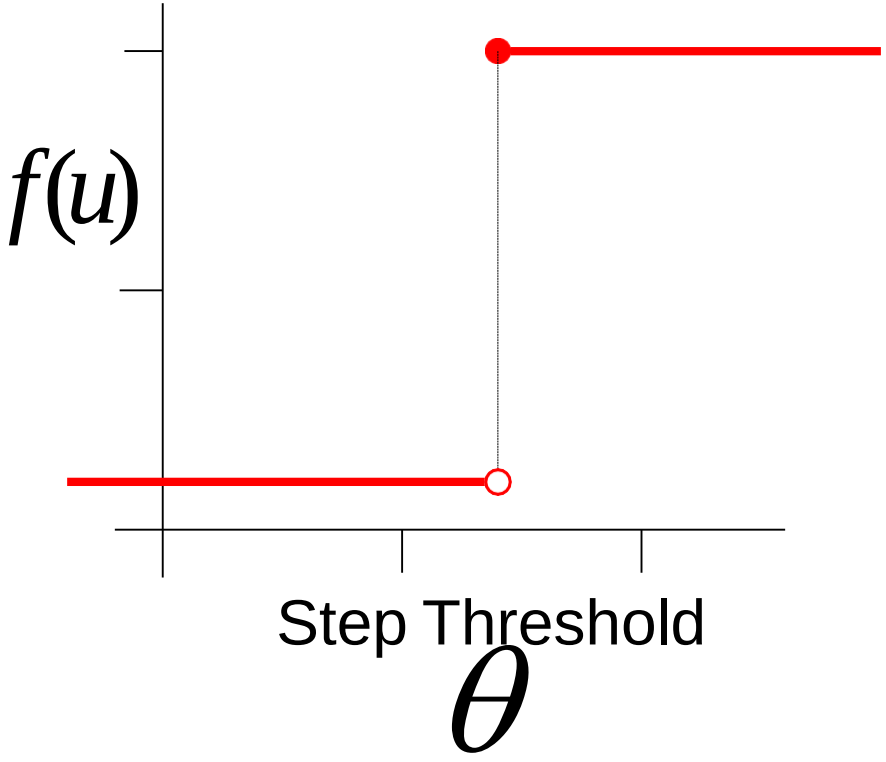
- for each example n
  1. Given input $\mathbf{x}^{(n)}$, propagate activity forward ($\mathbf{x}^{(n)} \rightarrow \mathbf{h}^{(n)} \rightarrow o^{(n)}$) (**forward pass**)
  2. Propagate gradients backward (**backward pass**)
  3. Update each weight (via gradient descent)

The Perceptron: Threshold Activation Function

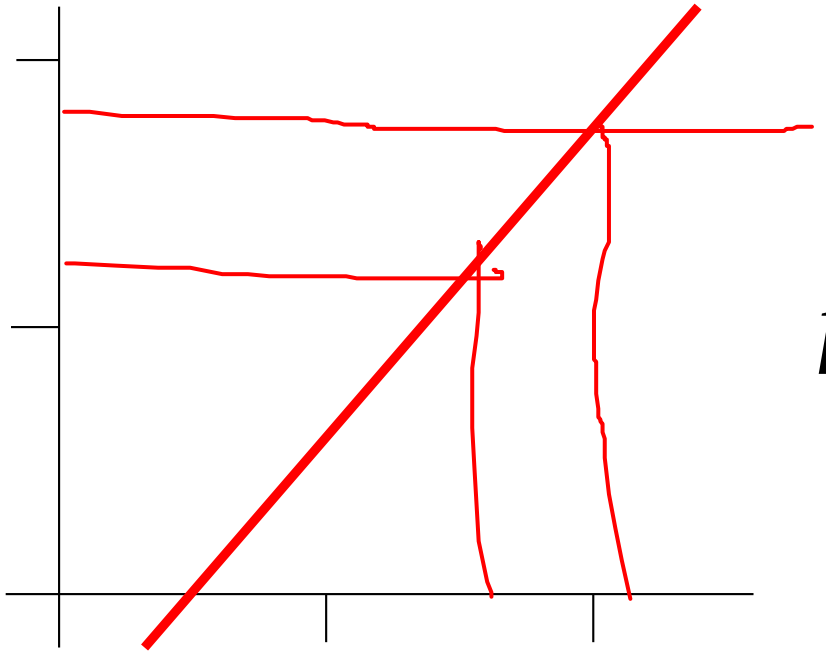
- Binary classifier functions
- Threshold activation function

$$u = \sum_{n=1}^{N} w_n x_n$$

$$f(u) = \begin{cases} 0 & u < \theta \\ 1 & u \geq \theta \end{cases}$$



Linear Activation functions



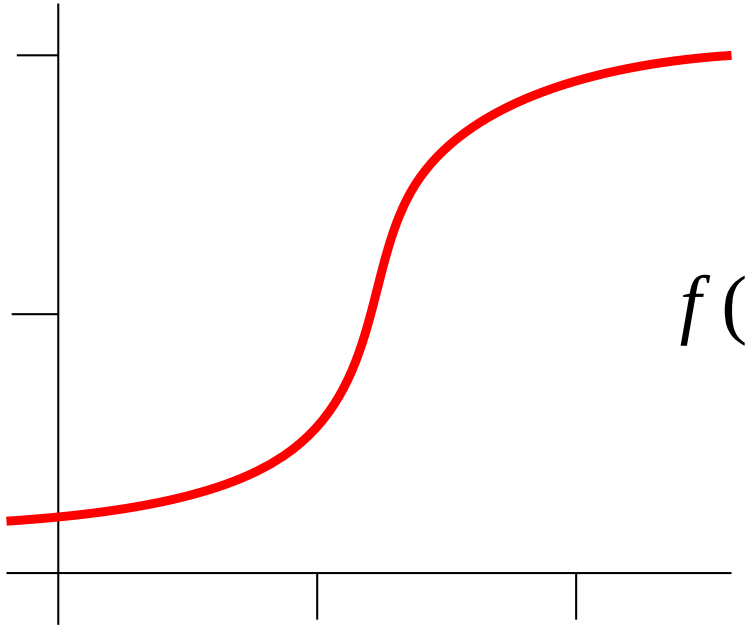
Linear

$$f(u) = u = \sum_{n=1}^{N} w_n x_n$$

Output is scaled sum of inputs

Nonlinear Activation Functions

Sigmoid Neuron unit function



$$f(u) = \frac{1}{1+e^{-u}}$$

Sigmoid

- **Linear Activation**

- The linear activation function, also known as "no activation," or "identity function" (multiplied x1.0), is where the activation is proportional to the input. **The function doesn't do anything to the weighted sum of the input, it simply spits out the value it was given.**

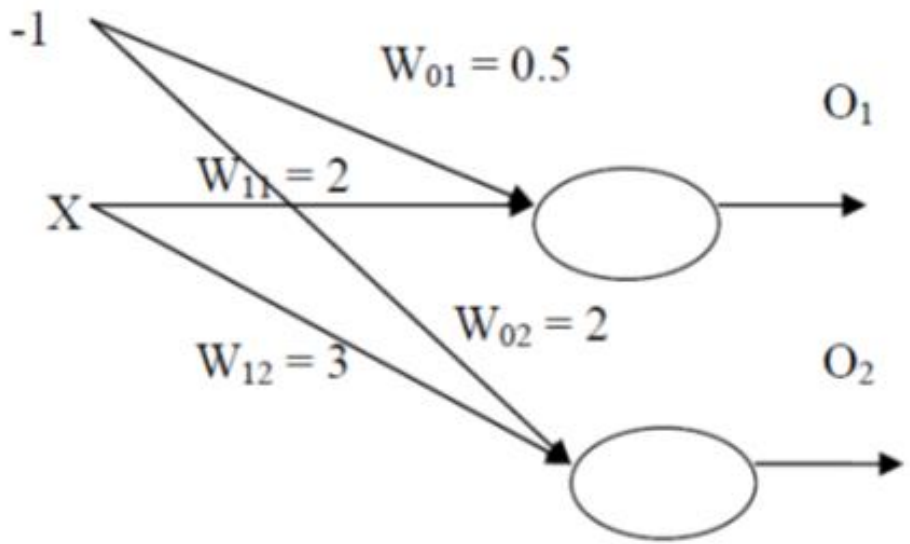
- **Generalized Delta Rule for updating weights**

$$\Delta w_{ij} = -c\,(\partial Error / \partial w_{ij}) = -c[-(d_j - O_j)\,.\,f'(act_j)\,.\,x_i]$$

- C is the learning rate
- F’ is the derivative of the activation function

For the following network, find the new weights by the generalized delta rule. Activation Function of the neuron is linear, i.e. y = activation. The learning rate (c) is 0.2. The training pair is [2; 3, 2]; i.e. input = 2 and output 1 = 3 and output 2 = 2. The current weights are shown on the link. There is only one input and two outputs.

Derivative of activation function is 1



$$\Delta w_{ij} = -c\,(\partial Error/\partial w_{ij}) = -c[-(d_j - O_j)\,.\,f'(act_j)\,.\,x_i]$$

Show the mathematical working of Artificial Neural Network by taking the case in figure below. First two columns are the input values for X1 and X2 and the third column is the desired output.

0	0	0
0	1	0
1	0	1
1	1	1

Learning rate = 0.2
Threshold = 0.5
Actual output = W1X1+W2X2

Next weight adjustment = Wn+ΔWn

Change in weight = ΔWn = learning rate * (desired output- output) * Xn

Show complete iterations for acquiring the desired output?

x1	x2	w1	w2	d	y	Δw1	Δw2

$y = x_1 w_1 + x_2 w_2 \mid \Delta w_1 = \eta (d - a) \cdot x_1 \mid w_{n1} = w_0 + \Delta w_n \mid \eta = 0.2 \quad \theta = 0.5$

$y = x_1 w_1 + x_2 w_2 \mid \Delta w_1 = \eta(d-a)\cdot x_1 \mid w_N = w_0 + \Delta w_N \mid \eta = 0.2 \qquad \theta = 0.5$

day / date:

x_1	x_2	w_1	w_2	d	y	Δw_1	Δw	R.W
0	0	1	1	0	0	0	0	$0.2(0-1)\cdot 1$
0	1	1	1	0	1	0	-0.2	-0.2
1	0	1	0.8	1	1	0	0	$1.8 > 0.5 = 1$
1	1	1	0.8	1	1	0	0	
0	0	1	0.8	0	0	0	0	$0.8 > 0.5$
0	1	1	0.8	0	1	0	-0.2	$0.2(0-1)\cdot 1$
1	0	1	0.6	1	1	0	0	-0.2
1	1	1	0.6	1	1	0	0	$1 + 0.6 = 1.6 > 0.5 = 1$
0	0	1	0.6	0	0	0	0	$0.6 > 0.5 = 1$
0	1	1	0.6	0	1	0	-0.2	
1	0	1	0.4	1	1	0	0	$1 > 0.5 = 1$
1	1	1	0.4	1	1	0	0	$1 + 0.4 = 1.4 > 0.5 = 1$
0	0	1	0.4	0	0	0	0	$0.4 < 0.5 = 0$
0	1	1	0.4	0	0	0	0	
1	0	1	0.4	1	1	0	0	$1 + .4 = 1.4 > 0.5 = 1$
1	1	1	0.4	1	1	0	0	

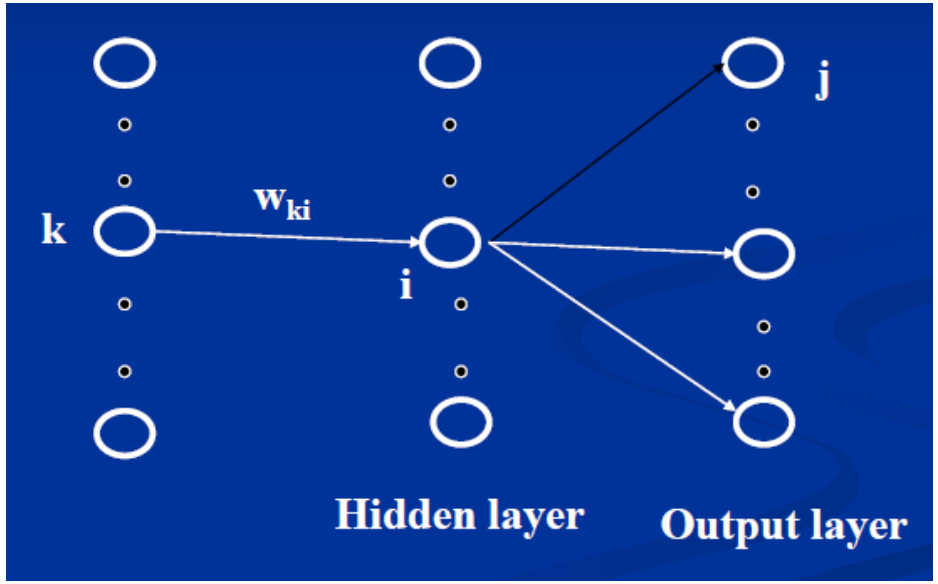
$(1-0) \quad y = 1 \times 1 + 0 \times 0.4 = 1 + 0 = 1 > 0.5 = 1 \qquad w_1 = 1, \; w_2 = 0.4$

$(0-1) \quad y = 0 \times 1 + 1 \times 0.4 = 0.4 < 0.5 = 0$

$(0-0) \quad y = 0 \times 1 + 0 \times 0.4 = 0$

$(1-1) \quad y = 1 \times 1 + 1 \times 0.4 = 1 + 0.4 = 1.4 > 0.5 = 1$

- **Generalized Delta Rule with hidden layers**

$$\Delta w_{ki} = -c\,(\partial \mathrm{Error} / \partial w_{ki})$$
$$= -c[-2\Sigma_j \{(d_j - O_j)\, f'(act_j)\, w_{ij}\}\, f'(act)_i\, x_k]$$

- C is the learning rate
- F' is the derivative of the activation function